SQLAlchemy 2.0 ORM & Alembic: Complete Engineering Notes

Targets SQLAlchemy 2.0.x, Alembic 1.13+, Python 3.10+. Examples use SQLite for zero setup and PostgreSQL for production.

Contents

Subsections marked *old vs new* compare the legacy 1.x way with the 2.0 way. Section 5.7 to 5.9 explain a complete real-world schema.

- 1. The Big Picture
- 2. Environment and Project Setup
- 3. The Engine
  - 3.1 Old vs new: engine usage
- 4. The Session
  - 4.5 Old vs new: Session usage
- 5. Defining Models
  - 5.7 Old vs new: declarative_base + Column vs DeclarativeBase + Mapped
  - 5.8 Worked example: three models line by line (UniqueConstraint, **table_args**, upsert)
  - 5.9 Import and type reference (UUID, BigInteger, Text, TIMESTAMP, func)
- 6. Relationships
  - 6.6 Old vs new: relationships
- 7. Querying with select()
  - 7.7 Old vs new: Query API vs select()
- 8. CRUD Service Layer
- 9. Transactions and Error Handling
- 10. Async SQLAlchemy (short version)
- 11. Alembic: Schema Migrations
- 12. What Autogenerate Detects and Misses
- 13. Common Errors and Fixes
- 14. Best Practices Checklist
- 15. Quick Cheat Sheet

---

1. The Big Picture

1.1 What problem does an ORM solve?

Databases speak in **tables, rows, columns, foreign keys**. Python code speaks in **classes, objects, attributes, references**. An **Object-Relational Mapper (ORM)** translates between the two so you write `user.email = 'x'` instead of hand-writing `UPDATE users SET email = 'x' WHERE id = 1`.

1.2 The layers (top to bottom)

Layer	Role	Key objects
Application	FastAPI, CLI, scripts	your code
ORM	Maps classes to tables, tracks object state	`DeclarativeBase`, `Mapped`, `Session`
Core	SQL expression language, schema objects	`select()`, `insert()`, `Table`, `text()`
Engine and Pool	Manages connections, dialect translation	`create_engine()`
DBAPI driver	Talks to the DB over the wire	`psycopg2`, `sqlite3`, `asyncpg`
Database	Stores the data	PostgreSQL, SQLite, MySQL

1.3 Core vs ORM

- **Core** is schema-centric: you build SQL with Python expressions and get rows back. Good for bulk or reporting work.
- **ORM** is domain-centric and built on Core: you work with objects, and the **Session** figures out what SQL to emit. Both share the same `select()` construct in 2.0.

1.4 Glossary

Term	Meaning
Engine	Factory and pool of DB connections, plus the SQL dialect. Created once per process.
Session	A short-lived workspace that holds objects and one transaction at a time.

Term	Meaning
Identity Map	Inside one session, one DB row maps to exactly one Python object.
Unit of Work (UoW)	The session collects changes in memory and writes them in an ordered batch on flush.
Metadata	A registry of all tables. Alembic compares it against the real DB.
Migration	A versioned script that moves the schema from revision A to B.

> **2.0 mindset:** use `select()` plus `session.execute()` / `session.scalars()`, and typed `Mapped[...]` models. The old `session.query(...)` still works but is legacy.

---

2. Environment and Project Setup

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install sqlalchemy alembic psycopg2-binary
pip freeze > requirements.txt
```

```
project_root/
  alembic/
    versions/          # generated migration scripts (commit these)
    env.py             # Alembic runtime config
    script.py.mako     # template for new migrations
  app/
    database.py        # engine, SessionLocal, get_db
    models.py          # ORM classes
    services.py        # CRUD / business logic
  main.py
  alembic.ini
  requirements.txt
```

Connection URL format

`dialect+driver://user:password@host:port/dbname`

Database	URL
SQLite (file)	`sqlite:///./app.db`
SQLite (memory)	`sqlite://`
PostgreSQL	`postgresql+psycopg2://user:pw@localhost:5432/mydb`
PostgreSQL async	`postgresql+asyncpg://user:pw@localhost/mydb`
MySQL	`mysql+pymysql://user:pw@localhost/mydb`

Keep credentials in environment variables, never in committed files.

---

3. The Engine

```
# database.py
import os
from sqlalchemy import create_engine

DATABASE_URL = os.getenv('DATABASE_URL', 'sqlite:///./app.db')

engine = create_engine(
    DATABASE_URL,
    echo=False,            # True logs all SQL; use only while debugging
    pool_pre_ping=True,    # test connections before use (survives DB
restarts)
    connect_args={'check_same_thread': False} if
DATABASE_URL.startswith('sqlite') else {},
)
```

Key points

- The engine does not hold one connection forever. It **lends** connections from a pool and takes them back.
- Create **one engine per database per process**, at module level.
- `check_same_thread=False` is a SQLite-only setting needed when a web framework uses multiple threads.

Pool option	Purpose
`pool_size`	Connections kept open (default 5)

Pool option	Purpose
`max_overflow`	Extra temporary connections above pool_size (default 10)
`pool_recycle`	Reopen connections older than N seconds (avoids server-side timeouts)
`pool_pre_ping`	Check liveness before handing out a connection

3.1 Old vs new: engine usage

Older way (1.x)	Current way (2.0)	Why it changed
`engine.execute('SELECT 1')`	`with engine.connect() as conn: conn.execute(text('SELECT 1'))`	`Engine.execute()` was removed. Raw SQL strings must be wrapped in `text()`.
`create_engine(url, future=True)`	`create_engine(url)`	`future=True` was a 1.4 bridge flag. In 2.0 that behavior is always on.
Connections auto-commit after each statement	Call `conn.commit()`, or use `with engine.begin() as conn:`	2.0 uses 'commit as you go': nothing is saved until you commit.
`MetaData(bind=engine)`	Pass the engine at call time: `metadata.create_all(engine)`	Bound metadata was removed.

```
from sqlalchemy import text

with engine.begin() as conn:          # commits on success, rolls back on
error
    conn.execute(text('UPDATE users SET name = :n WHERE id = :i'), {'n':
'Asha', 'i': 1})
```

---

4. The Session

4.1 Creating sessions

```
from collections.abc import Generator
from sqlalchemy.orm import sessionmaker, Session

SessionLocal = sessionmaker(bind=engine, autoflush=False,
expire_on_commit=False)

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()      # always release the connection to the pool
```

- `autoflush` (default True): pending changes are flushed automatically before a query. Setting it False gives you explicit control but means queries may not see unflushed changes.
- `expire_on_commit` (default True): after commit, all attributes are marked stale and reloaded on next access. Setting False lets you keep reading attributes after commit or after the session closes, at the cost of possibly stale data.

4.2 Object lifecycle states

State	Meaning
Transient	Created with `User(...)`, not in any session
Pending	Added with `session.add()`, not yet flushed
Persistent	Flushed or loaded, has a row and an identity in the session
Detached	Was persistent but the session closed or `expunge()` was called
Deleted	Marked for delete and flushed, gone after commit

4.3 flush vs commit vs rollback

Call	What it does
`flush()`	Sends pending SQL **inside the open transaction**. Not permanent. Use it to get auto-generated IDs early.
`commit()`	Flushes, then permanently ends the transaction. A new one begins on next use.
`rollback()`	Discards everything since the last commit and expires objects.
`refresh(obj)`	Re-SELECTs the object from the DB, overwriting in-memory values.
`expire(obj)`	Marks attributes stale so they reload on next access.

4.4 Usage patterns

```
# Pattern A: context manager, explicit commit
with SessionLocal() as session:
    session.add(User(name='Asha', email='asha@example.com'))
    session.commit()

# Pattern B: begin() commits on success, rolls back on exception
with SessionLocal() as session, session.begin():
    session.add(User(name='Ravi', email='ravi@example.com'))

# Pattern C: FastAPI dependency
# def endpoint(db: Session = Depends(get_db)): ...
```

Rules of thumb: one session per unit of work (one request, one job). Never share a session across threads. Never keep a session open globally.

4.5 Old vs new: Session usage

Older way (1.x)	Current way (2.0)	Note
`Session(autocommit=True)`	Not available	Autocommit mode was removed. `autocommit=False` is still accepted but redundant, so leave it out.
`session.query(User).get(1)`	`session.get(User, 1)`	`Query.get()` is legacy.
`session.begin(subtransactions=True)`	`session.begin_nested()`	Subtransactions were removed; use a SAVEPOINT.
`session.execute('SELECT ...')`	`session.execute(text('SELECT ...'))`	Plain strings are no longer accepted.
`sessionmaker(..., future=True)`	`sessionmaker(...)`	The `future` flag is unnecessary.
`scoped_session` as a global	One `Session` per request via a dependency (`get_db`)	`scoped_session` is still valid for thread-local code, but explicit sessions are easier to reason about.

For queries (`session.query(...)` vs `select(...)`), see section 7.7.

---

5. Defining Models

5.1 Base and a first model

```
# models.py
from datetime import datetime
from typing import Optional
from sqlalchemy import String, DateTime, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'

    id: Mapped[int] = mapped_column(primary_key=True)          #
```

```
autoincrement by default
    name: Mapped[str] = mapped_column(String(50))               # NOT NULL
(non-Optional)
    email: Mapped[str] = mapped_column(String(120), unique=True, index=True)
    job_title: Mapped[Optional[str]] = mapped_column(String(64))  # Optional
-> NULL allowed
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )

    def __repr__(self) -> str:
        return f'<User id={self.id} email={self.email!r}>'
```

How nullability works: `Mapped[str]` means NOT NULL, `Mapped[Optional[str]]` (or `str | None`) means nullable. You rarely need `nullable=` explicitly.

5.2 Python type to SQL type

Python annotation	Default SQL type
`int`	INTEGER
`str`	VARCHAR (give `String(n)` for a length)
`bool`	BOOLEAN
`float`	FLOAT
`datetime`	DATETIME / TIMESTAMP
`date`	DATE
`Decimal`	NUMERIC (use `Numeric(10, 2)` for money)
`dict` / `list`	use `JSON` (or `JSONB` on Postgres) explicitly
`enum.Enum` subclass	`Enum(...)`

5.3 Common `mapped_column` options

Option	Effect
`primary_key=True`	Primary key
`unique=True`	UNIQUE constraint
`index=True`	Creates an index

Option	Effect
`default=...`	**Python-side** default, applied by SQLAlchemy on INSERT
`server_default=...`	**Database-side** default, part of the DDL (`func.now()`, `text('0')`)
`onupdate=...`	Python-side value applied on UPDATE
`ForeignKey('users.id')`	Foreign key to another table

`default` vs `server_default`: server defaults also apply to rows inserted outside SQLAlchemy, and Alembic can see them.

5.4 Table-level constraints and indexes

```
from sqlalchemy import UniqueConstraint, CheckConstraint, Index

class Product(Base):
    __tablename__ = 'products'
    __table_args__ = (
        UniqueConstraint('sku', 'vendor_id', name='uq_products_sku_vendor'),
        CheckConstraint('price >= 0', name='ck_products_price_nonneg'),
        Index('ix_products_name_vendor', 'name', 'vendor_id'),
    )
    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(String(32))
    name: Mapped[str] = mapped_column(String(100))
    vendor_id: Mapped[int]
    price: Mapped[float]
```

5.5 Enums, mixins, naming conventions

```
import enum
from sqlalchemy import Enum, MetaData

class Status(str, enum.Enum):
    active = 'active'
    blocked = 'blocked'

# status: Mapped[Status] = mapped_column(Enum(Status), default=Status.active)

class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(server_default=func.now(),
onupdate=func.now())

# Class Order(TimestampMixin, Base): ...
```

Always set a naming convention. Without it, constraints get DB-generated names, and Alembic cannot reliably drop or alter them later.

```
convention = {
    'ix': 'ix_%(column_0_label)s',
    'uq': 'uq_%(table_name)s_%(column_0_name)s',
    'ck': 'ck_%(table_name)s_%(constraint_name)s',
    'fk': 'fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s',
    'pk': 'pk_%(table_name)s',
}

class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=convention)
```

5.6 Creating tables directly

```
Base.metadata.create_all(bind=engine)   # fine for prototypes and tests
```

`create_all` only creates **missing tables**. It never alters existing ones. Once you adopt Alembic, stop using it for real databases.

5.7 Old vs new: `declarative_base()` + `Column` vs `DeclarativeBase` + `Mapped`

You will meet two styles of model code. Both work in SQLAlchemy 2.0 and Alembic reads either one, because Alembic only looks at `Base.metadata`. **Pick one style per project.** New code should use the 2.0 style.

Aspect	Older style (1.x, 'classic declarative')	New style (2.0)
Base class	`Base = declarative_base()`	`class Base(DeclarativeBase): pass`
Where it is imported from	`sqlalchemy.ext.declarative` (old, deprecated) or `sqlalchemy.orm`	`sqlalchemy.orm`
Column declaration	`id = Column(Integer, primary_key=True)`	`id: Mapped[int] = mapped_column(primary_key=True)`
Nullability	Always spelled out: `nullable=False`	Inferred: `Mapped[str]` is NOT NULL, `Mapped[Optional[str]]` is nullable
SQL type	Required: `Column(Text, ...)`	Inferred from the Python type (`int` gives INTEGER, `str` gives VARCHAR); pass a type only when you need a specific one (`Text`, `BigInteger`)
Editor / mypy support	`obj.id` is seen as a `Column` object	`obj.id` is seen as a real `int`
Relationships	`relationship('Book', backref='author')`	`Mapped[list['Book']] = relationship(back_populates='author')`
Plain class vs dataclass	Plain class only	Optional `MappedAsDataclass` mixin for generated `__init__`

The same model, both ways

Older style (the style of your code):

```
import uuid
from sqlalchemy import BigInteger, Column, Integer, Text, TIMESTAMP
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import declarative_base
from sqlalchemy.sql import func

Base = declarative_base()

class PullRequest(Base):
    __tablename__ = 'pull_requests'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    repo_full_name = Column(Text, nullable=False)
    pr_number = Column(Integer, nullable=False)
    head_sha = Column(Text, nullable=False)
    installation_id = Column(BigInteger, nullable=False)
    status = Column(Text, nullable=False, default='pending')
    created_at = Column(TIMESTAMP(timezone=True), server_default=func.now(),
nullable=False)
```

New style (2.0):

```
import uuid
from datetime import datetime
from sqlalchemy import BigInteger, Text, TIMESTAMP, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class PullRequest(Base):
    __tablename__ = 'pull_requests'

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True,
default=uuid.uuid4)
    repo_full_name: Mapped[str] = mapped_column(Text)
    pr_number: Mapped[int]                                  # INTEGER NOT
NULL, nothing else needed
    head_sha: Mapped[str] = mapped_column(Text)
    installation_id: Mapped[int] = mapped_column(BigInteger)  # int alone
would give INTEGER (32-bit)
    status: Mapped[str] = mapped_column(Text, default='pending')
    created_at: Mapped[datetime] = mapped_column(TIMESTAMP(timezone=True),
server_default=func.now())
```

What changed, line by line

- `Column(UUID(as_uuid=True), ...)` became `Mapped[uuid.UUID]`. In 2.0, `uuid.UUID` maps to the generic `Uuid` type: a native UUID column on PostgreSQL and a CHAR(32) on SQLite. You no longer need the PostgreSQL-specific import unless you want that exact type.
- `nullable=False` disappeared everywhere, because none of the annotations are `Optional`.
- `Mapped[int]` is INTEGER, so a 64-bit ID still needs `BigInteger` passed to `mapped_column`.
- `Mapped[str]` alone gives VARCHAR with no length. Pass `Text` when you want an unlimited text column.

5.8 Worked example: three models line by line

This section takes a real schema (a pull-request review bot) and explains every construct in it. Three tables: `pull_requests` (one row per reviewed PR), `findings` (issues found on a PR), and `patterns` (recurring issue text per repository).

Table	Role	Keys and constraints
`pull_requests`	One row per reviewed PR	UUID primary key
`findings`	Issues found on a PR (many per PR)	UUID primary key, foreign key `pr_id` to `pull_requests.id`
`patterns`	Recurring issue text, counted per repo	UUID primary key, UNIQUE on (`repo_full_name`, `pattern_text`)

What each construct means

Code	Meaning
`UUID(as_uuid=True)`	PostgreSQL UUID type. `as_uuid=True` returns Python `uuid.UUID` objects instead of strings.
`default=uuid.uuid4`	**Python-side default.** Pass the function itself (no parentheses); SQLAlchemy calls it once per INSERT. The database never sees it, so a row inserted from `psql` gets no ID.
`Text`	Unlimited-length string. On PostgreSQL it performs the same as `VARCHAR(n)`, so use it when you have no real length limit.

Code	Meaning
`Integer` vs `BigInteger`	32-bit (up to about 2.1 billion) vs 64-bit (up to about 9.2 quintillion). IDs that come from outside systems (GitHub installation IDs) should be `BigInteger`.
`nullable=False`	Adds `NOT NULL`.
`TIMESTAMP(timezone=True)`	`TIMESTAMP WITH TIME ZONE`. Store timezone-aware values (UTC) so comparisons stay correct.
`server_default=func.now()`	The **database** fills in `now()` on INSERT. Alembic can see it, and it also works for rows inserted outside Python.
`ForeignKey('pull_requests.id')`	A string of `table.column`, not the class name. It creates the FOREIGN KEY constraint.
`UniqueConstraint(...)`	A table-level UNIQUE constraint over one or more columns.
`__table_args__`	A tuple of table-level configuration (constraints, indexes, options) passed to the underlying table.

`server_default` and reading the value back: after an INSERT, a database-generated value like `created_at` is fetched automatically on PostgreSQL (via `RETURNING`). On other databases call `session.refresh(obj)` to load it.

`UniqueConstraint` and `__table_args__` in detail

```
__table_args__ = (
    UniqueConstraint('repo_full_name', 'pattern_text',
name='uq_repo_pattern'),
)
```

- `__table_args__` is a special class attribute that SQLAlchemy reads when building the table. Anything that belongs to the **table** rather than to one column goes here.
- A one-item tuple **needs the trailing comma**, otherwise it is just parentheses around an object and SQLAlchemy raises an error.
- The last element may be a dict of table options, for example `(UniqueConstraint(...), {'comment': 'Recurring review issues'})`.
- The generated SQL is `CONSTRAINT uq_repo_pattern UNIQUE (repo_full_name, pattern_text)`.
- **Always give constraints a `name`.** Alembic needs it to drop or alter the constraint later.

The constraint is on the **pair**, not on each column. Each column on its own may repeat:

repo_full_name	pattern_text	Allowed?
acme/api	missing null check	Yes
acme/api	unused import	Yes, same repo but different text
acme/web	missing null check	Yes, same text but different repo
acme/api	missing null check	**No**, the pair already exists: `IntegrityError`

A composite unique constraint also creates an index on the pair. Lookups by `repo_full_name` alone can use it, because it is the leading column. Lookups by `pattern_text` alone cannot.

Improvements worth making

1. **Clean the imports.** `declarative_base` is imported twice, and `String` and `relationship` are imported but unused.
2. **Name classes in the singular.** `Findings` should be `Finding` (class singular, table plural).
3. **Make `pr_id` NOT NULL with a delete rule.** As written it is nullable and has no `ondelete`, so findings can be orphaned. Use `ForeignKey('pull_requests.id', ondelete='CASCADE')`.
4. **Index foreign keys.** PostgreSQL does not index them automatically. Add `index=True` to `pr_id`.
5. **Add `onupdate=func.now()` to `updated_at`.** A `server_default` only applies on INSERT, so the timestamp would never change.
6. **Constrain `status` and `severity`.** Free text invites typos. Use an `Enum` or a `CheckConstraint`.
7. **Add relationships** so you can write `pr.findings` and `finding.pull_request`.
8. **Increment `frequency` in the database** (`frequency + 1`), not by reading and writing in Python, which races under concurrency.

The same schema, modern and improved

```
import uuid
from datetime import datetime
from sqlalchemy import BigInteger, ForeignKey, Text, TIMESTAMP,
UniqueConstraint, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column,
relationship
```

```
class Base(DeclarativeBase):
    pass

class PullRequest(Base):
    __tablename__ = 'pull_requests'

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True,
default=uuid.uuid4)
    repo_full_name: Mapped[str] = mapped_column(Text)
    pr_number: Mapped[int]
    head_sha: Mapped[str] = mapped_column(Text)
    installation_id: Mapped[int] = mapped_column(BigInteger)
    status: Mapped[str] = mapped_column(Text, default='pending')
    created_at: Mapped[datetime] = mapped_column(TIMESTAMP(timezone=True),
server_default=func.now())

    findings: Mapped[list['Finding']] = relationship(
        back_populates='pull_request', cascade='all, delete-orphan',
passive_deletes=True
    )

class Finding(Base):
    __tablename__ = 'findings'

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True,
default=uuid.uuid4)
    pr_id: Mapped[uuid.UUID] = mapped_column(
        ForeignKey('pull_requests.id', ondelete='CASCADE'), index=True
    )
    file: Mapped[str] = mapped_column(Text)
    line: Mapped[int]
    severity: Mapped[str] = mapped_column(Text)
    message: Mapped[str] = mapped_column(Text)
    agent: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(TIMESTAMP(timezone=True),
server_default=func.now())

    pull_request: Mapped['PullRequest'] =
relationship(back_populates='findings')

class Pattern(Base):
    __tablename__ = 'patterns'
    __table_args__ = (
        UniqueConstraint('repo_full_name', 'pattern_text',
name='uq_repo_pattern'),
    )
```

```
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True,
default=uuid.uuid4)
    repo_full_name: Mapped[str] = mapped_column(Text)
    pattern_text: Mapped[str] = mapped_column(Text)
    agent: Mapped[str] = mapped_column(Text)
    frequency: Mapped[int] = mapped_column(default=1)
    updated_at: Mapped[datetime] = mapped_column(
        TIMESTAMP(timezone=True), server_default=func.now(),
onupdate=func.now()
    )
```

Using the unique constraint: upsert for `patterns`

The composite constraint is exactly what PostgreSQL's `ON CONFLICT` needs. Insert the pattern if it is new, otherwise bump its counter, in one atomic statement:

```
from sqlalchemy.dialects.postgresql import insert as pg_insert

stmt = pg_insert(Pattern).values(repo_full_name=repo, pattern_text=text_,
agent=agent)
stmt = stmt.on_conflict_do_update(
    constraint='uq_repo_pattern',        # or index_elements=
['repo_full_name', 'pattern_text']
    set_={'frequency': Pattern.frequency + 1, 'updated_at': func.now()},
)
session.execute(stmt)
session.commit()
```

This is also why the constraint needs a stable `name`: the statement refers to it.

5.9 Import and type reference

Every import in the example, explained

Import	Comes from	What it does
`uuid`	Python standard library	`uuid.uuid4()` makes a random UUID; `uuid.UUID` is the Python type for it

Import	Comes from	What it does
`Column`	`sqlalchemy`	Older way to declare a column. Replaced by `mapped_column` in the 2.0 style.
`Integer`, `BigInteger`, `String`, `Text`, `TIMESTAMP`	`sqlalchemy`	SQL column types (table below)
`ForeignKey`	`sqlalchemy`	Declares a foreign key to another table's column
`UniqueConstraint`	`sqlalchemy`	Table-level UNIQUE over one or more columns
`UUID`	`sqlalchemy.dialects.postgresql`	PostgreSQL-specific UUID type. In 2.0 the generic `sqlalchemy.Uuid` (or just `Mapped[uuid.UUID]`) works on every database.
`declarative_base`	`sqlalchemy.orm`	Older factory that returns the base class. Replaced by `DeclarativeBase`.
`relationship`	`sqlalchemy.orm`	Links two models so you can navigate objects (section 6)
`func`	`sqlalchemy` (also reachable as `sqlalchemy.sql.func`)	Calls SQL functions: `func.now()`, `func.count()`, `func.lower(col)`

Generic vs dialect types. Types imported from `sqlalchemy` are *generic*: they adapt to whatever database you use. Types from `sqlalchemy.dialects.postgresql` (`JSONB`, `ARRAY`, `UUID`, `INET`) are *dialect-specific* and give you database-only features at the cost of portability.

SQL column types you will use most

Type	SQL it produces	Use it for
`String(n)`	`VARCHAR(n)`	Text with a real maximum (codes, short names)
`Text`	`TEXT`	Unbounded text (messages, descriptions, SHAs)
`SmallInteger`	`SMALLINT` (up to 32,767)	Small counters and codes
`Integer`	`INTEGER` (up to about 2.1 billion)	Normal counters, line numbers, small IDs
`BigInteger`	`BIGINT` (up to about 9.2 quintillion)	IDs from external systems, very large counters
`Numeric(p, s)`	`NUMERIC(p, s)`	Money and exact decimals (never `Float`)
`Float`	`FLOAT`	Approximate measurements
`Boolean`	`BOOLEAN`	Flags
`Date`, `Time`	`DATE`, `TIME`	Calendar dates, clock times
`DateTime` / `TIMESTAMP`	`TIMESTAMP`	Moments in time. Both accept `timezone=True`; on PostgreSQL they behave the same. Prefer `TIMESTAMP(timezone=True)`.
`JSON` / `JSONB`	`JSON` / `JSONB`	Flexible documents. `JSONB` (PostgreSQL) can be indexed and queried.
`Uuid`	`UUID` or `CHAR(32)`	Globally unique identifiers
`LargeBinary`	`BYTEA` / `BLOB`	Raw bytes
`Enum(...)`	`ENUM` or `VARCHAR` + check	A fixed set of values

UUID or auto-increment integer for primary keys?

	Integer (auto-increment)	UUID (`uuid.uuid4`)
Size	4 or 8 bytes	16 bytes

	Integer (auto-increment)	UUID (`uuid.uuid4`)
Generated by	The database	Your application (known before INSERT) or the database
Safe to expose in URLs	Guessable and enumerable	Not guessable
Merging data across databases	Collisions likely	Collisions practically impossible
Index behavior	Sequential, compact, fast	Random order, larger index, more page splits
Debugging	Easy to read	Harder to read

UUIDs suit APIs, webhooks, and distributed writers. Integers suit internal tables that are never exposed. If you want the **database** to generate UUIDs so inserts outside Python also work, use `server_default=text('gen_random_uuid()')` (built in from PostgreSQL 13) in addition to or instead of `default=uuid.uuid4`.

---

6. Relationships

A **ForeignKey** lives in the database. A **relationship()** lives in Python and gives you object navigation. They work together.

6.1 One-to-many / many-to-one

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class Author(Base):
    __tablename__ = 'authors'
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(80))
    books: Mapped[list['Book']] = relationship(back_populates='author',
cascade='all, delete-orphan')

class Book(Base):
    __tablename__ = 'books'
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    author_id: Mapped[int] = mapped_column(ForeignKey('authors.id',
ondelete='CASCADE'))
    author: Mapped['Author'] = relationship(back_populates='books')
```

`back_populates` links both sides: `book.author = a` automatically updates `a.books` in memory.

6.2 One-to-one

Use `uselist=False` on the parent side, or annotate with a scalar `Mapped['Profile']`, plus a **unique** FK on the child.

6.3 Many-to-many (plain)

```
from sqlalchemy import Table, Column

book_tags = Table(
    'book_tags', Base.metadata,
    Column('book_id', ForeignKey('books.id'), primary_key=True),
    Column('tag_id', ForeignKey('tags.id'), primary_key=True),
)

class Tag(Base):
    __tablename__ = 'tags'
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(40), unique=True)

# on Book:
# tags: Mapped[list[Tag]] = relationship(secondary=book_tags)
```

If the link needs extra data (quantity, role, timestamps), use an **association object**: a full model with two FKs plus extra columns, with relationships on each side.

6.4 Cascades

Cascade	Meaning
`save-update` (default)	Adding a parent also adds children
`delete`	Deleting a parent deletes children
`delete-orphan`	A child removed from the parent's collection is deleted
`all`	Everything except `delete-orphan`

ORM `cascade` works in Python. `ondelete='CASCADE'` on the FK works in the database. Using both is common; add `passive_deletes=True` to let the DB do the work efficiently.

6.5 Loading strategies and the N+1 problem

N+1: loading 100 authors, then touching `author.books` in a loop, fires 1 query plus 100 more. Fix it by choosing a loader.

Strategy	SQL behavior	Best for
`lazy='select'` (default)	Separate query on first access	Rarely-used relationships
`selectinload`	One extra `SELECT ... WHERE id IN (...)`	**Collections** (one-to-many, many-to-many)
`joinedload`	LEFT JOIN in the same query	**Scalar** many-to-one / one-to-one
`subqueryload`	Extra query with a subquery	Rarely needed now
`raiseload`	Raises an error if lazy-loaded	Catching accidental N+1 in dev

```
from sqlalchemy.orm import selectinload, joinedload

stmt = select(Author).options(selectinload(Author.books))
authors = session.scalars(stmt).all()

stmt = select(Book).options(joinedload(Book.author))
```

Detached error: accessing a lazy relationship after the session is closed raises `DetachedInstanceError`. Eager-load what you need inside the session.

6.6 Old vs new: relationships

Older way (1.x)	Current way (2.0)	Note
`books = relationship('Book', backref='author')`	`books: Mapped[list['Book']] = relationship(back_populates='author')` plus the matching attribute on `Book`	`backref` creates the reverse side invisibly. `back_populates` makes both sides explicit, typed, and visible to editors. `backref` still works.
`author_id = Column(Integer, ForeignKey('authors.id'))`	`author_id: Mapped[int] = mapped_column(ForeignKey('authors.id'))`	Same change as in 5.7.
`lazy='dynamic'` for huge collections	`lazy='write_only'` with `WriteOnlyMapped[Book]`	`dynamic` is legacy. `write_only` never loads the collection implicitly; you query it with `select()`.
`.options(joinedload('books'))` (string names)	`.options(joinedload(Author.books))` (class attributes)	String-based loader options were removed in 2.0.
Plain relationship, no type hint	`Mapped[list['Book']]` or `Mapped['Author']`	Tells type checkers what comes back; `Optional` marks a nullable many-to-one.

---

7. Querying with `select()`

7.1 Basics

```
from sqlalchemy import select, func, and_, or_, desc

stmt = select(User).where(User.email == 'a@b.com')
user = session.scalars(stmt).first()

# multiple conditions
stmt = select(User).where(User.name.like('A%'), User.job_title.is_not(None))
stmt = select(User).where(or_(User.name == 'A', User.name == 'B'))

# ordering, paging
stmt = select(User).order_by(desc(User.created_at)).offset(20).limit(10)

# membership, ranges
stmt = select(User).where(User.id.in_([1, 2, 3]))
stmt = select(User).where(User.created_at.between(start, end))
```

7.2 Getting results out

Method	Returns
`session.get(User, 5)`	One object by primary key, or `None` (checks identity map first)
`session.scalars(stmt).all()`	List of ORM objects
`.first()`	First object or `None`
`.one()`	Exactly one, else raises `NoResultFound` / `MultipleResultsFound`
`.one_or_none()`	One or `None`, raises if more than one
`session.scalar(stmt)`	First column of first row (good for counts)
`session.execute(stmt).all()`	List of `Row` tuples (use when selecting multiple columns)

7.3 Joins, aggregates, grouping

```
# join via relationship or explicit condition
stmt = select(Book.title, Author.name).join(Author, Book.author_id ==
Author.id)
stmt = select(Book).join(Book.author).where(Author.name == 'Asha')

# aggregate
count = session.scalar(select(func.count()).select_from(User))

# group by + having
stmt = (
    select(Author.name, func.count(Book.id).label('n'))
    .join(Book).group_by(Author.id).having(func.count(Book.id) > 3)
)

# exists / subquery
sub = select(Book.id).where(Book.author_id == Author.id).exists()
stmt = select(Author).where(sub)
```

7.4 Bulk UPDATE / DELETE without loading objects

```
from sqlalchemy import update, delete

session.execute(update(User).where(User.job_title.is_(None)).values(job_title
='Unassigned'))
session.execute(delete(User).where(User.id < 10))
session.commit()
```

These bypass per-object tracking. They are much faster, but in-memory objects may be stale afterward.

7.5 Bulk insert and upsert

```
from sqlalchemy import insert
session.execute(insert(User), [
    {'name': 'A', 'email': 'a@x.com'},
    {'name': 'B', 'email': 'b@x.com'},
])

# PostgreSQL upsert
from sqlalchemy.dialects.postgresql import insert as pg_insert
stmt = pg_insert(User).values(name='A', email='a@x.com')
stmt = stmt.on_conflict_do_update(index_elements=['email'], set_={'name':
stmt.excluded.name})
session.execute(stmt)
```

7.6 Raw SQL (escape hatch)

```
from sqlalchemy import text
rows = session.execute(text('SELECT * FROM users WHERE id = :id'), {'id':
1}).all()
```

Always use **bound parameters** (`:id`), never string formatting. That prevents SQL injection.

7.7 Old vs new: `Query` API vs `select()`

The older `session.query(...)` API still runs in 2.0 but is considered legacy. Do not mix it with `select()` in new code. Here is each task in both styles:

Task	Older way (`Query`)	Current way (`select`)
All rows	`session.query(User).all()`	`session.scalars(select(User)).all()`
Filter, first match	`session.query(User).filter(User.id == 1).first()`	`session.scalars(select(User).where(User.id == 1)).first()`
Filter by keyword	`session.query(User).filter_by(name='A')`	`select(User).filter_by(name='A')` (still exists) or `.where(User.name == 'A')`
By primary key	`session.query(User).get(1)`	`session.get(User, 1)`
Count	`session.query(User).count()`	`session.scalar(select(func.count()).select_from(User))`
Join	`session.query(Book).join(Author)`	`select(Book).join(Author)`
Order and limit	`.order_by(User.id).limit(10)`	same methods on `select(User)`
Existence check	`session.query(exists().where(...)).scalar()`	`session.scalar(select(exists().where(...)))`
Bulk update	`session.query(User).filter(...).update({...})`	`session.execute(update(User).where(...).values(...))`

Task	Older way (`Query`)	Current way (`select`)
Bulk delete	`session.query(User).filter(...).delete()`	`session.execute(delete(User).where(...))`
Eager load	`.options(joinedload('books'))`	`.options(joinedload(Author.books))`
Raw SQL	`session.execute('SELECT ...')`	`session.execute(text('SELECT ...'))`
Several columns	`session.query(User.id, User.name).all()`	`session.execute(select(User.id, User.name)).all()`

Why the change: `select()` is the same construct used by Core, so there is one way to build queries. It is built first and executed separately, which makes statements easy to reuse, compose, and test.

---

8. CRUD Service Layer

```
# services.py
from typing import Optional, Sequence
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
from models import User

class DuplicateEmail(Exception): ...

def create_user(db: Session, name: str, email: str, job_title: Optional[str]
= None) -> User:
    user = User(name=name, email=email, job_title=job_title)
    db.add(user)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()                # session is unusable until rolled back
        raise DuplicateEmail(email)
    db.refresh(user)                 # load server-generated values
(created_at)
    return user

def get_user(db: Session, user_id: int) -> Optional[User]:
    return db.get(User, user_id)

def list_users(db: Session, skip: int = 0, limit: int = 100) ->
Sequence[User]:
    return
db.scalars(select(User).order_by(User.id).offset(skip).limit(limit)).all()
```

```
def update_email(db: Session, user_id: int, new_email: str) ->
Optional[User]:
    user = db.get(User, user_id)
    if user is None:
        return None
    user.email = new_email            # change is tracked automatically
(dirty)
    db.commit()
    return user

def delete_user(db: Session, user_id: int) -> bool:
    user = db.get(User, user_id)
    if user is None:
        return False
    db.delete(user)
    db.commit()
    return True
```

Design tip: decide where commits live. Either services commit (simple apps) or the caller/request layer commits once (better when one request touches several services).

---

9. Transactions and Error Handling

- Every session is always inside a transaction. The first statement starts it. `commit()` or `rollback()` ends it.
- After a failed flush (for example `IntegrityError`) the session is in a failed state. You **must** call `rollback()` before reuse.
- **Savepoints:** `with session.begin_nested():` creates a SAVEPOINT. A failure inside rolls back only that block.

```
for row in rows:
    try:
        with session.begin_nested():
            session.add(User(**row))
    except IntegrityError:
        pass                  # skip duplicate, keep the rest
session.commit()
```

- **Isolation level:** set per engine with `create_engine(url, isolation_level='REPEATABLE READ')` when you need stronger guarantees.
- **Row locking:** `select(User).where(...).with_for_update()` for read-modify-write safety on PostgreSQL.

---

10. Async SQLAlchemy (short version)

```
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker,
AsyncSession

engine = create_async_engine('postgresql+asyncpg://user:pw@localhost/db')
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_user(user_id: int):
    async with AsyncSessionLocal() as session:
        return await session.get(User, user_id)
```

- Use an async driver (`asyncpg` , `aiosqlite`).
- Lazy loading does not work in async. **Eager-load** relationships with `selectinload` or `joinedload` .
- `expire_on_commit=False` is strongly recommended here.
- Alembic itself runs synchronously. Use `alembic init -t async alembic` for the async env template.

---

11. Alembic: Schema Migrations

11.1 Why Alembic?

`create_all` cannot change existing tables. Real projects need **versioned, repeatable, reversible** schema changes across dev, staging, and production. Alembic stores migrations as Python scripts forming a chain (a revision graph), and records the current position in a table called `alembic_version` inside your database.

11.2 Setup

```
alembic init alembic
```

This creates `alembic.ini` and the `alembic/` folder. Now edit `alembic/env.py` (recommended over hard-coding the URL in the ini file):

```
# alembic/env.py (key parts)
import os
from app.models import Base            # import ALL models so metadata is
populated

config = context.config
config.set_main_option('sqlalchemy.url', os.environ['DATABASE_URL'])

target_metadata = Base.metadata

# inside run_migrations_online(), configure with:
context.configure(
    connection=connection,
    target_metadata=target_metadata,
    compare_type=True,                 # detect column type changes
    compare_server_default=True,       # detect default changes
    render_as_batch=True,              # needed for SQLite ALTER support
)
```

Critical: if a model module is never imported, its tables are missing from `Base.metadata`, and autogenerate will think they should be dropped.

11.3 The daily workflow

1. Change your models.
2. Generate: `alembic revision --autogenerate -m 'add job_title to users'`
3. **Read and edit** the generated file. Autogenerate is a draft, not gospel.
4. Apply: `alembic upgrade head`
5. Commit the migration file with the code change.

11.4 Anatomy of a migration file

```
"""create users table

Revision ID: a1b2c3d4e5f6
Revises: None
"""
from alembic import op
import sqlalchemy as sa

revision = 'a1b2c3d4e5f6'
down_revision = None          # points to the parent revision
branch_labels = None
depends_on = None
```

```
def upgrade() -> None:
    op.create_table(
        'users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(length=50), nullable=False),
        sa.Column('email', sa.String(length=120), nullable=False),
        sa.Column('job_title', sa.String(length=64), nullable=True),
        sa.Column('created_at', sa.DateTime(timezone=True),
                  server_default=sa.text('now()'), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('email'),
    )
    op.create_index('ix_users_email', 'users', ['email'])

def downgrade() -> None:
    op.drop_index('ix_users_email', table_name='users')
    op.drop_table('users')
```

The `revision` and `down_revision` pair forms the chain. Always write a working `downgrade()` so you can roll back.

11.5 Command reference

Goal	Command
Create empty migration	`alembic revision -m 'message'`
Autogenerate migration	`alembic revision --autogenerate -m 'message'`
Apply everything pending	`alembic upgrade head`
Apply next N	`alembic upgrade +1`
Upgrade to a specific revision	`alembic upgrade a1b2c3`
Roll back one	`alembic downgrade -1`
Roll back everything	`alembic downgrade base`
Where is the DB now?	`alembic current`
Show chain	`alembic history --verbose`
Show heads (should be one)	`alembic heads`
Check models vs DB drift	`alembic check`

Goal	Command
Mark revision without running it	`alembic stamp head`
Print SQL instead of executing	`alembic upgrade head --sql`
Merge two heads	`alembic merge -m 'merge' <rev1> <rev2>`

11.6 Common `op` operations

Operation	Example
Create / drop table	`op.create_table(...)`, `op.drop_table('t')`
Add / drop column	`op.add_column('users', sa.Column('age', sa.Integer()))`, `op.drop_column('users', 'age')`
Alter column	`op.alter_column('users', 'name', type_=sa.String(100), nullable=False)`
Rename column	`op.alter_column('users', 'old', new_column_name='new')`
Rename table	`op.rename_table('old', 'new')`
Index	`op.create_index('ix_x', 'users', ['col'])`, `op.drop_index('ix_x', 'users')`
Unique / check	`op.create_unique_constraint('uq_x', 'users', ['col'])`
Foreign key	`op.create_foreign_key('fk_x', 'books', 'authors', ['author_id'], ['id'])`
Drop constraint	`op.drop_constraint('uq_x', 'users', type_='unique')`
Raw SQL	`op.execute('...')`

11.7 Safe pattern: adding a NOT NULL column to a table with data

Adding a NOT NULL column with no default fails if rows already exist. Do it in three steps.

```
def upgrade():
    op.add_column('users', sa.Column('status', sa.String(20), nullable=True))
# 1. nullable
    op.execute(sa.text("UPDATE users SET status = 'active' WHERE status IS
NULL"))  # 2. backfill
    op.alter_column('users', 'status', nullable=False)
# 3. tighten
```

Alternatively, use `server_default='active'` in one step.

11.8 Data migrations

Alembic autogenerates **DDL only** (structure). To transform rows, write DML by hand in `upgrade()`. For anything non-trivial, define lightweight table stubs inside the migration instead of importing your app models, because models change over time but a migration must stay valid forever.

```
users = sa.table('users', sa.column('id', sa.Integer), sa.column('job_title',
sa.String))
op.execute(users.update().where(users.c.job_title.is_(None)).values(job_title
='Unassigned'))
```

11.9 Renames: the data-loss trap

Alembic sees a renamed column as *drop old plus add new*, which **destroys the data**. Always inspect the generated script and replace the pair with:

```
op.alter_column('users', 'fullname', new_column_name='name')
```

The same applies to renaming tables (`op.rename_table`).

11.10 SQLite and `batch_alter_table`

SQLite cannot ALTER most things (drop column, change type, add constraint). Alembic's **batch mode** works around this by creating a new table, copying data, dropping the old one, and renaming.

```
with op.batch_alter_table('users') as batch_op:
    batch_op.alter_column('name', type_=sa.String(100))
    batch_op.drop_column('legacy_field')
```

Setting `render_as_batch=True` in `env.py` makes autogenerate emit this automatically. Constraints must be **named** (see the naming convention in 5.5) for batch mode to manage them.

11.11 PostgreSQL ENUMs and custom types

- Creating a table with an `Enum` column also creates the PG type.
- **Adding a value** to an existing enum is not autogenerated. Write it by hand, and note it cannot run inside a transaction on older PG versions:

```
with op.get_context().autocommit_block():
    op.execute("ALTER TYPE status ADD VALUE IF NOT EXISTS 'archived'")
```

- On `downgrade`, drop the type explicitly with `sa.Enum(name='status').drop(op.get_bind())`.
- Triggers, functions, views, and extensions all need manual `op.execute()`.

11.12 Adopting Alembic on an existing database

1. Make models match the current DB schema.
2. `alembic revision --autogenerate -m 'baseline'` (then review).
3. `alembic stamp head` on the existing DB to mark it as current **without running** the baseline.
4. From now on, use normal migrations.

11.13 Branches and merges

If two developers each create a migration from the same parent, you get **two heads** and `upgrade head` fails. Fix with `alembic merge -m 'merge heads' <rev1> <rev2>`, which creates an empty migration with two parents. Run `alembic heads` in CI to catch this early.

11.14 Team and production practices

- Migrations are **immutable** once merged and applied elsewhere. Never edit them; add a new one.
- One logical change per migration, with a clear message.
- Run `alembic upgrade head` as a deploy step, **before** starting new application code.
- Make migrations **backward compatible** with the old code (expand, then migrate, then contract) for zero-downtime deploys.
- Test migrations: upgrade, then downgrade, then upgrade on a copy of production-like data.
- Large tables: adding indexes may lock writes. On PostgreSQL use `postgresql_concurrently=True` inside `autocommit_block()`.
- Use `alembic upgrade head --sql` to give a DBA reviewable SQL.

---

12. What Autogenerate Detects and Misses

Detected	NOT detected (handle manually)
Table add / remove	Table or column **renames**
Column add / remove	Changes to **CHECK** constraints without names
Nullable changes	Adding **values to PG enums**
Column type changes (with `compare_type=True`)	Triggers, functions, views, sequences
Index and unique constraint changes	Changes to Python-side `default=` (not in the DB)
Foreign key changes	Data content changes
Server default changes (with `compare_server_default=True`)	Models in modules that were never imported

---

13. Common Errors and Fixes

Error	Cause	Fix
`DetachedInstanceError`	Lazy attribute accessed after session closed	Eager-load (`selectinload`) or keep session open
`PendingRollbackError`	Previous flush failed, session not rolled back	Call `session.rollback()`
`IntegrityError`	Unique / FK / NOT NULL violation	Validate first or catch, then rollback
`MultipleResultsFound`	`.one()` or `.scalar_one()` matched >1 row	Add filters, or use `.first()`
`Multiple head revisions`	Parallel migration branches	`alembic merge`
`Target database is not up to date`	Pending migrations exist	`alembic upgrade head`

Error	Cause	Fix
`Can't locate revision`	DB references a deleted or missing migration file	Restore file or `alembic stamp` the correct revision
Autogenerate emits `drop_table` for everything	Models not imported in `env.py`	Import all model modules
Empty migration generated	`target_metadata` missing or DB already matches	Verify `target_metadata = Base.metadata`
`ProgrammingError` on `ALTER` in SQLite	SQLite lacks ALTER support	Use `batch_alter_table`
SQLite threading error	Connection used from another thread	`check_same_thread=False`

---

14. Best Practices Checklist

SQLAlchemy

- One engine per process, one session per request or unit of work, always closed.
- Typed `Mapped[...]` models, with a metadata naming convention.
- Use `select()` and `scalars()`. Avoid legacy `session.query`.
- Eager-load collections with `selectinload`. Consider `raiseload` in dev to expose N+1.
- Bound parameters only; never build SQL strings with f-strings.
- Use `server_default` for defaults that should live in the database.
- Prefer bulk `insert` / `update` / `delete` statements for large operations.
- `echo=False` in production. Use real logging and slow-query monitoring.

Alembic

- Always review autogenerated output before applying.
- Never use `create_all` on environments managed by Alembic.
- Keep `downgrade()` functional, and keep exactly one head.
- Do not import app models into data migrations; use table stubs.
- Keep secrets out of `alembic.ini`; read from the environment in `env.py`.

---

15. Quick Cheat Sheet

```
# --- setup ---
engine = create_engine(url, pool_pre_ping=True)
SessionLocal = sessionmaker(engine, expire_on_commit=False)

# --- create ---
with SessionLocal() as s:
    s.add(User(name='A', email='a@x.com'));  s.commit()

# --- read ---
s.get(User, 1)
s.scalars(select(User).where(User.name == 'A')).all()
s.scalars(select(Author).options(selectinload(Author.books))).all()

# --- update / delete ---
u.email = 'new@x.com';  s.commit()
s.delete(u);  s.commit()
s.execute(update(User).where(...).values(...))

# --- transaction control ---
s.flush();  s.commit();  s.rollback();  s.refresh(u)
```

```
# --- alembic ---
alembic init alembic
alembic revision --autogenerate -m 'msg'
alembic upgrade head
alembic downgrade -1
alembic current && alembic history --verbose && alembic heads
alembic stamp head
```

Mental model in one line: *Models define the desired schema; Alembic moves the real database toward it, one reviewed step at a time; the Session reads and writes the data inside transactions.*